

Repository:	shayanmondal / 60-Day-DevSecOps-Engineer
Phase covered:	Phase 4 — CI/CD (Day 15 to Day 20)
Tools used:	GitHub Actions, Docker, GitHub Environments
Deliverables:	4 hands-on GitHub Actions pipelines (Project 1–4)

PHASE 4 — CI/CD

Day 15 — CI/CD fundamentals

- **CI (Continuous Integration)** — every code change is automatically built and tested as soon as it's pushed, catching problems early.
- **CD (Continuous Delivery/Deployment)** — automates how that tested code actually gets released to users.
- **Runner** — the machine (hosted or self-hosted) that actually executes the pipeline.
- **Job** — a unit of work in a pipeline, made up of ordered steps.
- **Stage / Step** — a single task inside a job, run in sequence.
- **Artifact** — a build output — binaries, reports, or packaged code — saved for later use or download.
- **Environment** — a target for the code, like development, staging, or production, each with its own config and access.

Day 16 — GitHub Actions deeper

- **Trigger** — the event that starts a workflow — a push, a pull request, a schedule, or a manual run.
- **Matrix** — runs the same job multiple times with different inputs (versions, OSes) in parallel.
- **Caching** — stores dependencies between runs so they don't have to be re-downloaded every time, speeding up builds.
- **Reusable workflow** — splitting build and test into separate jobs so pipelines stay modular and easy to reuse.

Day 17 — CI quality gates

- **Unit tests** — automated checks that individual pieces of code behave correctly.
- **Linting** — automated checks for code style and common mistakes.
- **Coverage** — how much of the codebase is actually exercised by tests.
- **Failure condition** — if any required check fails, the pipeline halts — stopping bad code from being merged or deployed.

Day 18 — Container CI/CD

- **Build image** — a CI job packages the app by building a Docker image directly from source.
- **Tag strategy** — giving each image a unique, traceable identity — e.g. the Git commit SHA or a version number.
- **Registry authentication** — logging in securely to the container registry where images are stored.
- **Push workflow** — uploading the image using least-privilege credentials — only the minimum access needed.

Day 19 — Deployment strategies

- **Dev / staging / prod** — a change moves through progressively more “real” environments before reaching users.
- **Approvals** — a manual sign-off gate before a risky step (like a production release) proceeds.
- **Rollback** — reverting to the last known-good version if a release goes wrong.
- **Blue/green** — keeping two identical environments and instantly switching traffic from old to new (with old kept as instant rollback).
- **Canary** — releasing a change to a small subset of users first, expanding only if it proves stable.

Day 20 — End-to-end pipeline

- **Full flow** — PR → test → lint → build → image scan placeholder → image publish → staging deploy → rollback.
- **Why it matters** — combines every earlier concept into one complete, portfolio-ready pipeline that mirrors how real teams ship software.

REPOSITORY OVERVIEW

All four projects live in the same repository, each with its own workflow file under **.github/workflows**, demonstrating a progressive build-up of CI/CD skills across Phase 4.

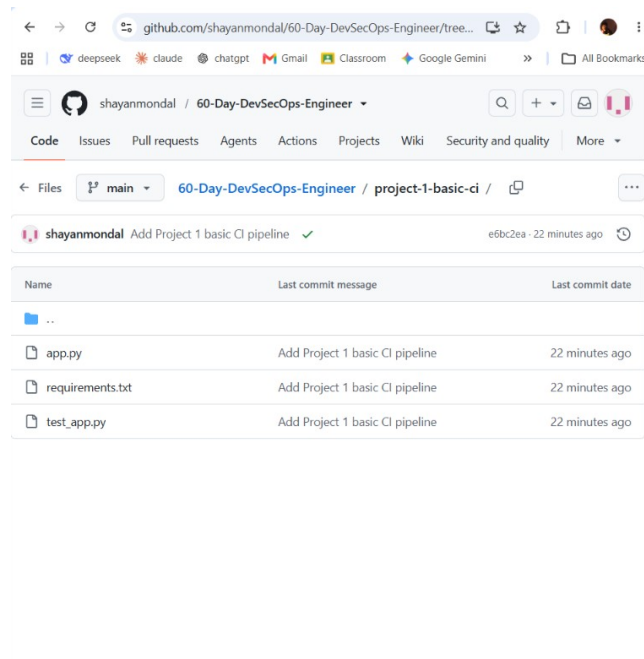


Figure 1 — Repository root showing Phase 1–4 project folders and documentation.

Objective: Build a first end-to-end CI workflow — run automated tests, produce a build artifact, and inspect the job logs.

What was built: A GitHub Actions workflow (project1-ci.yml) triggered on push. The **test** job checks out the code, sets up Python, installs dependencies, runs the test suite, creates a build artifact, and uploads it.

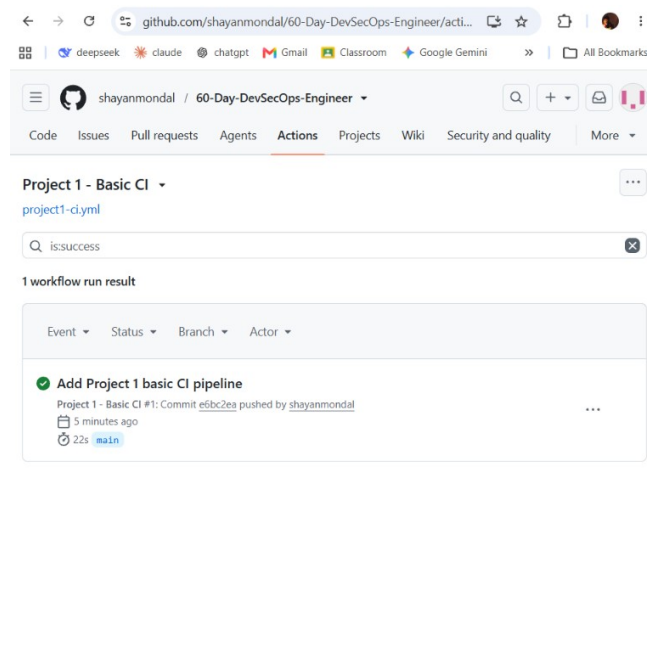


Figure 2 — Workflow run history showing a successful pipeline execution.

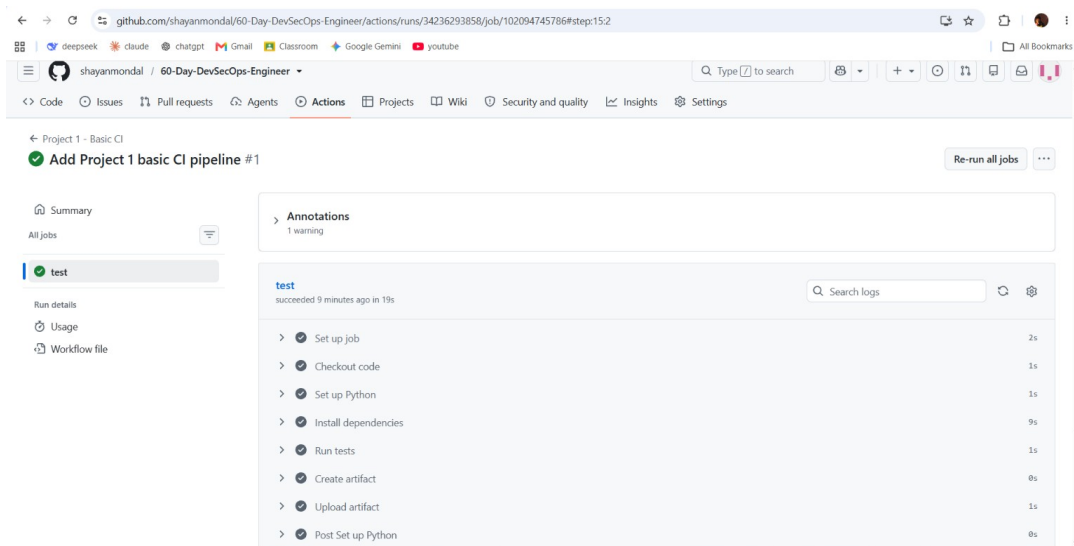


Figure 3 — Job steps: checkout, environment setup, dependency install, tests, artifact creation and upload — all passing.

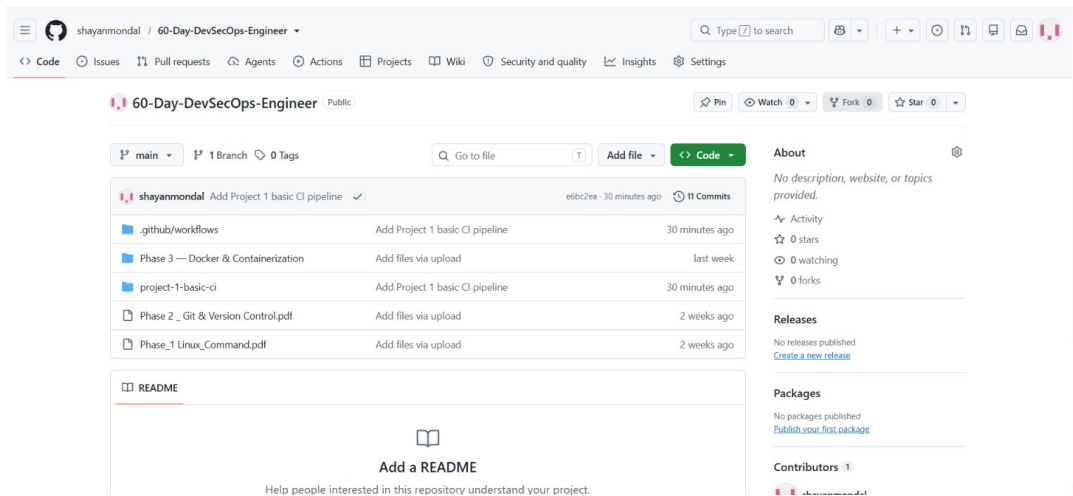


Figure 4 — Repository contents for project-1-basic-ci (app.py, requirements.txt, test_app.py).

Outcome: Successfully explained and built a basic CI pipeline that runs tests and publishes an artifact on every push.

Objective: Make tests and linting mandatory checks that block the pipeline on failure, implementing a CI quality gate.

What was built: A workflow (project2-quality-gate.yml) with two parallel jobs — **test** and **lint** — both required to pass before the change is considered good.

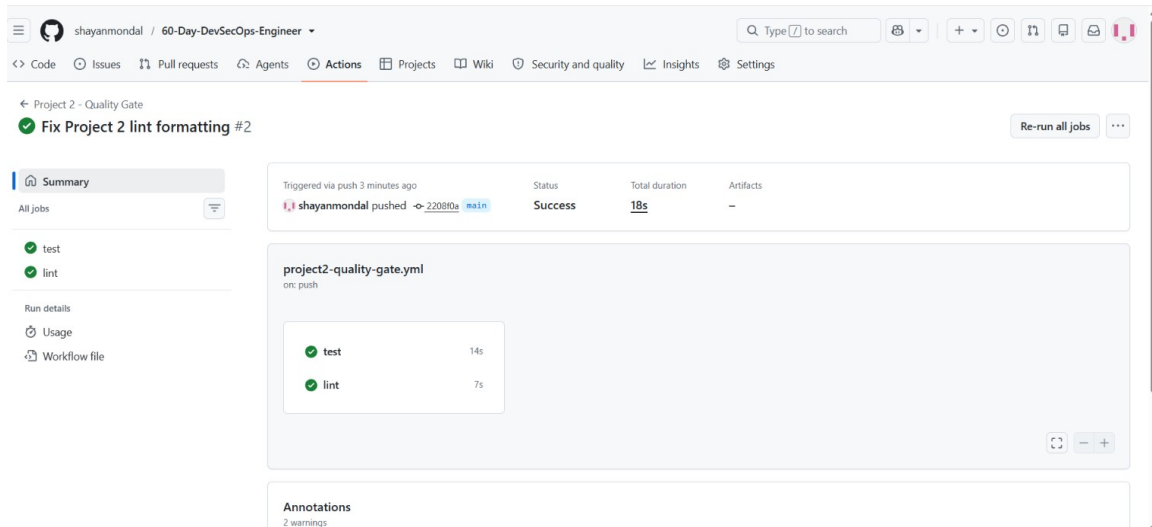


Figure 5 — Project 2 pipeline: test and lint jobs both completed successfully.

Outcome: Implemented working quality gates that enforce test and lint checks before code is accepted.

Objective: Automate building a Docker image in CI as the foundation for reproducible container publishing.

What was built: A workflow (container-ci.yml) with a **build** job that builds the application's Docker image on every push, laying the groundwork for tagging and registry push.

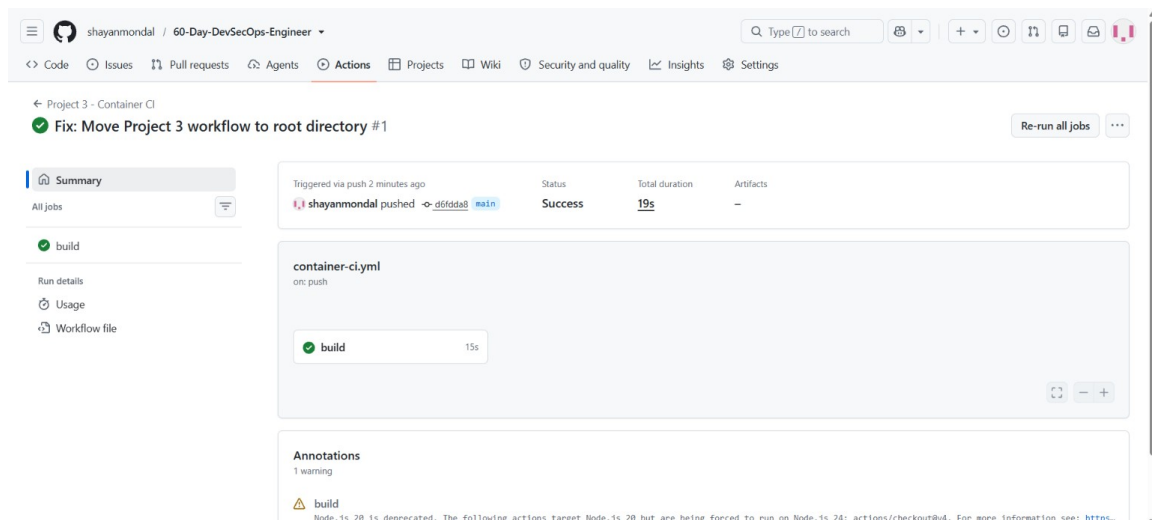


Figure 6 — Project 3 container build job completing successfully.

Outcome: Demonstrated an automated, reproducible container build step inside a CI pipeline.

Objective: Combine testing, building, and deployment into one complete pipeline — the Phase 4 capstone project.

What was built: A workflow (project4-deployment.yml) chaining three jobs: **test-and-lint** → **build-image** → **deploy-staging**, modeling a real PR-to-production flow with a staging environment.

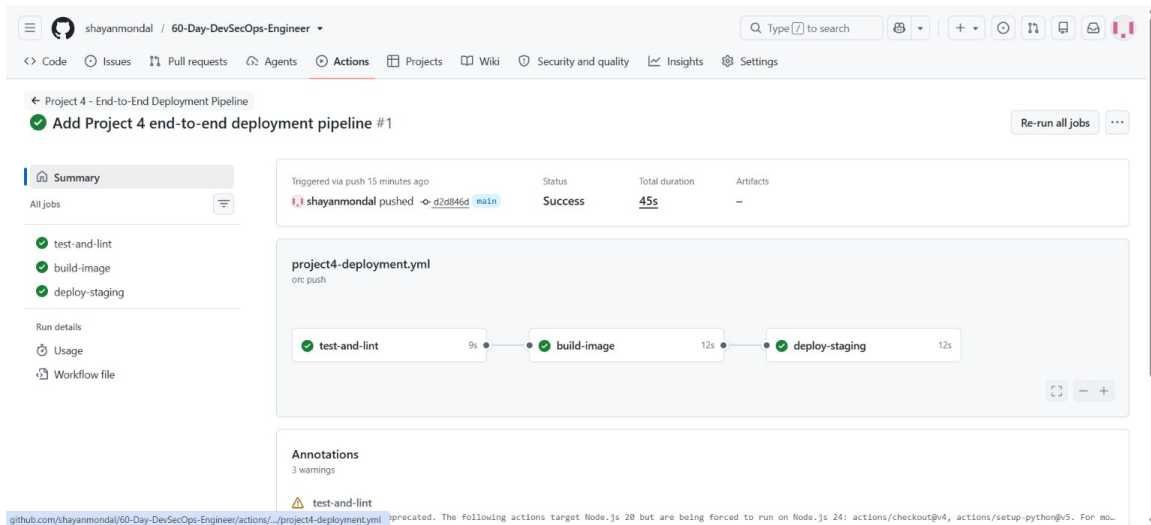


Figure 7 — Project 4 pipeline: test-and-lint, build-image, and deploy-staging jobs, all completed in 45s.

Outcome: A complete, portfolio-ready CI/CD pipeline covering the full Day 15–20 objectives — from automated testing and quality gates through containerization to staged deployment.

SUMMARY

Project	Day(s)	Focus	Key jobs
1 — Basic CI	15	CI fundamentals	test
2 — Quality Gate	17	Tests + linting as gates	test, lint
3 — Container CI	18	Docker image build in CI	build
4 — End-to-End Deployment	19–20	Full pipeline to staging	test-and-lint, build-image, deploy-staging

Across Projects 1–4, the Phase 4 objectives were met end-to-end: CI fundamentals (runners, jobs, artifacts) → enforced quality gates (tests + lint) → automated container builds → a full deployment pipeline with staging — forming a working CI/CD foundation suitable for portfolio demonstration.